

METHODOLOGY OVERVIEW

The proposed upgrade planning framework consists of two major stages: candidate link selection and upgrade plan evaluation. The candidate-selection stage first ranks all source–destination pairs according to their observed blocking probability. For each ranked SD pair, the candidate paths ($K = 3$) are ranked according to their path blocking probability, where a higher path blocking probability indicates a higher level of congestion. Based on the congestion strategy, candidate links are selected and passed to the PlanChecker framework for performance evaluation and cost feasibility analysis.

Path-Based Upgrade Criterion: In the path-based congestion strategy, the most congested candidate path is identified for each source–destination pair, and all links along that path are selected as candidate upgrade links.

Link-Level Upgrade Criterion: In the link-based congestion strategy, the most congested candidate path is first identified, after which the utilization of each link on that path is computed. The highest-utilization link is then selected as the upgrade candidate.

Plan_Checker Framework: The selected candidate links are initially assumed to be upgraded to their next feasible upgrade type based on the chosen upgrade strategy. The traffic matrix is projected to the next six-month upgrade cycle, followed by a one-month validation simulation to estimate the resulting blocking probability.

If the projected blocking probability exceeds the target threshold, PlanChecker progressively escalates the upgrade level of the selected links until the blocking constraint is satisfied or no further upgrades are feasible. The resulting upgrade set is then evaluated against budget and downtime constraints.

If the candidate set violates feasibility constraints, links are ranked by blocking contribution and a pruning process retains the most impactful upgrades while maintaining feasibility. Once a valid solution is obtained, any remaining budget and downtime are utilized for further strengthening. The final accepted plan is therefore the strongest performance-feasible and budget-compliant upgrade configuration for the current planning cycle.

Greedy Upgrade Strategy: The greedy strategy applies aggressive capacity expansion by prioritizing larger upgrade steps. A link initially operating in a single-core C-band configuration is first upgraded by converting all parallel fibers to a three-core configuration, while enabling C+L transmission only on the initially lit fiber. Subsequent upgrades activate the remaining dark fibers directly in the three-core C+L configuration until the link reaches its maximum feasible capacity.

Maximum-Capacity Upgrade Strategy: The maximum-capacity strategy upgrades a selected link directly to its maximum technology state in a single step. This includes converting the link to three-core operation and enabling C+L transmission across all available lit resources, thereby maximizing capacity expansion immediately.

Adaptive Planning Strategy: The adaptive planning strategy performs sequential and state-aware upgrades. Depending on the current network state, upgrades may include band extension, activation of dark fibers, or core expansion. This strategy incrementally increases capacity while maintaining flexibility in upgrade decisions.

Random Upgrade Strategy: The random strategy selects eligible links and feasible upgrade technologies randomly from the candidate pool. It serves as a baseline for comparison against the structured upgrade planning approaches.

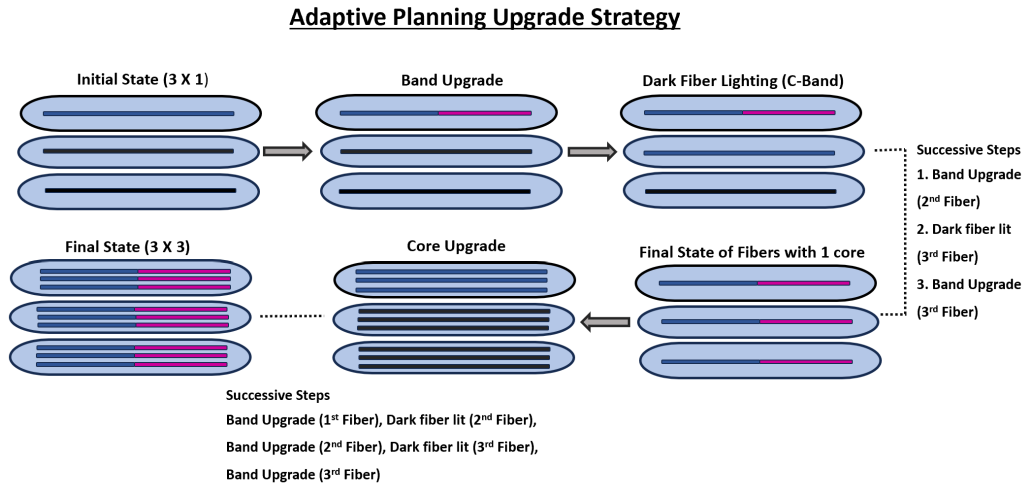
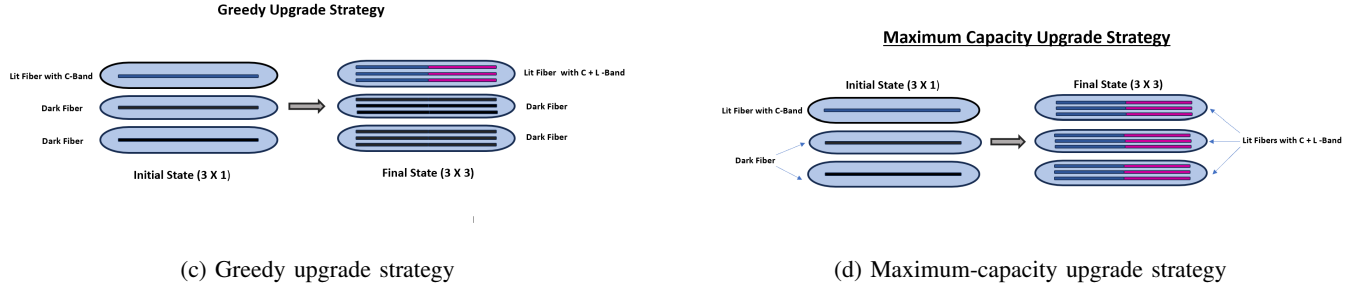
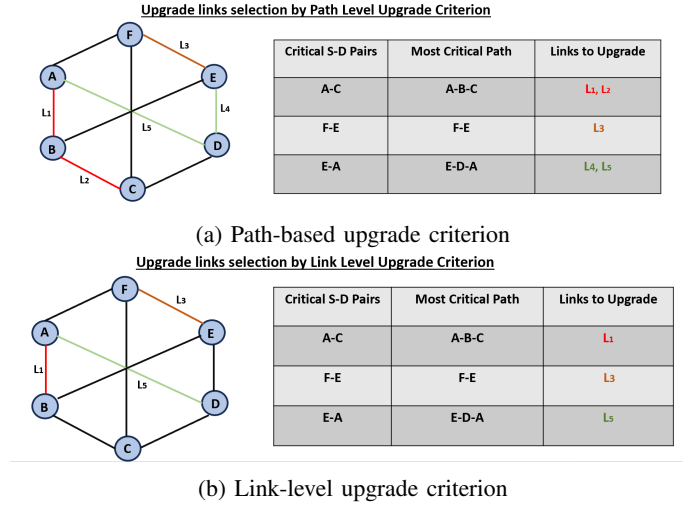


Fig. 1: Candidate selection and technology upgrade strategies used in the proposed framework.

Algorithm 1 Path-Based Upgrade Criterion

1: **Input:** SD blocking probabilities P_{SD} , candidate paths $\mathcal{P}_{sd}$, path statistics, link states
2: **Output:** Candidate upgrade link set $\mathcal{L}_{sel}$
3: Rank all source–destination pairs in descending order of P_{SD}
4: Initialize $\mathcal{L}_{sel} \leftarrow \emptyset$
5: **for** each source–destination pair (s, d) in ranked order **do**
6: Retrieve candidate paths $\mathcal{P}_{sd} = \{p_1, p_2, p_3\}$
7: **for** each path $p_k \in \mathcal{P}_{sd}$ **do**
8: Compute path blocking probability
$$BP(p_k) = \frac{\text{blocked requests on } p_k}{\text{total arrived requests on } p_k}$$

9: **end for**
10: Rank candidate paths in descending order of $BP(p_k)$
11: Select the most congested path p^*
12: **for** each link $e \in p^*$ **do**
13: **if** e is not at maximum upgrade state **then**
14: Add e to $\mathcal{L}_{sel}$
15: **end if**
16: **end for**
17: **end for**
18: Pass $\mathcal{L}_{sel}$ to PlanChecker for performance evaluation and cost feasibility analysis

Algorithm 2 Link-Based Upgrade Criterion

1: **Input:** SD blocking probabilities P_{SD} , path statistics, candidate paths $\mathcal{P}_{sd}$, link states
2: **Output:** Candidate upgrade link set $\mathcal{L}_{sel}$
3: Rank all SD pairs in descending order of P_{SD}
4: Initialize $\mathcal{L}_{sel} \leftarrow \emptyset$
5: **for** each SD pair (s, d) in ranked order **do**
6: Retrieve candidate paths $\mathcal{P}_{sd} = \{p_1, p_2, p_3\}$
7: **for** each path $p_k \in \mathcal{P}_{sd}$ **do**
8: Compute path blocking probability
$$BP(p_k) = \frac{\text{blocked requests on } p_k}{\text{total arrived attempts on } p_k}$$

9: **end for**
10: Rank candidate paths in descending order of $BP(p_k)$
11: Select the most congested path p^*
12: **for** each link $e \in p^*$ **do**
13: Compute link utilization
$$U(e) = \frac{\text{occupied spectrum slots on } e}{\text{total available spectrum slots on } e}$$

14: **end for**
15: Select the highest-utilization link
$$e^* = \arg \max_{e \in p^*} U(e)$$

16: **if** e^* is not at maximum upgrade state **then**
17: Add e^* to $\mathcal{L}_{sel}$
18: **end if**
19: **end for**
20: Pass $\mathcal{L}_{sel}$ to PlanChecker for performance evaluation and cost feasibility analysis

Algorithm 3 PlanChecker-Based Upgrade Framework

```
1: Input: Candidate links  $\mathcal{L}_{sel}$ , budget  $B$ , downtime limit  $T$ ,  
   blocking threshold  $B_{tr}$ , upgrade decision rule  $A$   
2: Output: Final accepted upgrade plan  $\mathcal{D}_{acc}$   
3: Initialize accepted plan  $\mathcal{D}_{acc} \leftarrow \emptyset$   
4: Generate initial upgrade plan  $\mathcal{D}_0$  for all links in  $\mathcal{L}_{sel}$  using  
   rule  $A$   
5: if  $\mathcal{D}_0$  satisfies budget and downtime constraints then  
6:   Evaluate blocking probability  $BP(\mathcal{D}_0)$   
7:    $\mathcal{D}_{acc} \leftarrow \mathcal{D}_0$   
8: else  
9:   Rank candidate links according to their blocking con-  
   tribution  
10:  Build the largest feasible subset satisfying budget and  
   downtime constraints  
11:  Set  $\mathcal{D}_{acc}$  to the feasible subset  
12:  Evaluate  $BP(\mathcal{D}_{acc})$   
13: end if  
14: while  $BP(\mathcal{D}_{acc}) > B_{tr}$  and higher upgrades exist do  
15:    $\mathcal{D}_{trial} \leftarrow \mathcal{D}_{acc}$   
16:   for each link  $e \in \mathcal{L}_{sel}$  do  
17:     Select the next higher upgrade technology for  $e$   
     using rule  $A$   
18:     Add the stronger decision to  $\mathcal{D}_{trial}$   
19:   end for  
20:   if  $\mathcal{D}_{trial}$  satisfies budget and downtime constraints  
   then  
21:     Evaluate  $BP(\mathcal{D}_{trial})$   
22:      $\mathcal{D}_{acc} \leftarrow \mathcal{D}_{trial}$   
23:   else  
24:     Keep previous accepted plan  
25:   break  
26: end if  
27: end while  
28: if  $BP(\mathcal{D}_{acc}) \leq B_{tr}$  then  
29:   while budget and downtime constraints allow further  
   strengthening do  
30:      $\mathcal{D}_{trial} \leftarrow \mathcal{D}_{acc}$   
31:     for each link  $e \in \mathcal{L}_{sel}$  do  
32:       Select the next higher upgrade technology for  
        $e$  using rule  $A$   
33:       if adding this decision keeps budget and down-  
       time feasible then  
34:         Add the stronger decision to  $\mathcal{D}_{trial}$   
35:       else  
36:         Keep previous decision for link  $e$   
37:       end if  
38:     end for  
39:     if  $\mathcal{D}_{trial}$  satisfies budget, downtime, and  
      $BP(\mathcal{D}_{trial}) \leq B_{tr}$  then  
40:        $\mathcal{D}_{acc} \leftarrow \mathcal{D}_{trial}$   
41:     else  
42:       break  
43:     end if  
44:   end while  
45: end if  
46: Return  $\mathcal{D}_{acc}$ 
```

Algorithm 4 Greedy Upgrade Decision Rule

```
1: Input: Link  $e$ , current state of  $e$   
2: Output: Next greedy upgrade decision for  $e$   
3: if  $e$  is in the initial single-core C-band configuration then  
4:   Convert all parallel fibers of  $e$  to three-core configu-  
   ration  
5:   Enable C+L transmission on the initially lit fiber  
6: else if there exists an unlit fiber then  
7:   Light the next dark fiber directly with three-core C+L  
   configuration  
8: else  
9:   No higher greedy upgrade exists  
10: end if
```

Algorithm 5 Maximum-Capacity Upgrade Decision Rule

```
1: Input: Link  $e$ , current state of  $e$   
2: Output: Maximum-capacity upgrade decision for  $e$   
3: if  $e$  is not at maximum capacity then  
4:   Upgrade  $e$  directly to the maximum technology state  
5:   Convert the selected fibers to three-core operation  
6:   Enable C+L bands on all lit cores  
7:   Set link  $e$  to maximum capacity state  
8: else  
9:   No higher upgrade exists  
10: end if
```

Algorithm 6 Adaptive Planning Upgrade Decision Rule

```
1: Input: Link  $e$ , current state of  $e$   
2: Output: Next sequential upgrade decision for  $e$   
3: if a lit fiber on  $e$  has only C-band then  
4:   Add L-band to the selected lit fiber  
5: else if there exists an unlit fiber and the current expansion  
   policy is C-band first then  
6:   Light the next dark fiber with C-band  
7: else if core upgrade is feasible then  
8:   Upgrade from single-core to three-core operation  
9: else  
10:  No higher sequential upgrade exists  
11: end if
```

Algorithm 7 Random Upgrade Decision Rule

- 1: **Input:** Candidate links $\mathcal{L}_{sel}$, current network state
 - 2: **Output:** Randomly selected link and upgrade decision
 - 3: Build candidate pool from eligible non-maximum links in $\mathcal{L}_{sel}$
 - 4: Randomly select a link e from the candidate pool
 - 5: Build feasible technology set $\mathcal{U}_e$ for link e
 - Feasible technologies may include:
 - band upgrade on a lit C-only fiber
 - lighting a dark fiber with C-band
 - core upgrade if the link is still single-core
 - 6: **if** $\mathcal{U}_e$ is not empty **then**
 - 7: Randomly select one upgrade $u \in \mathcal{U}_e$
 - 8: Return (e, u)
 - 9: **else**
 - 10: No feasible random upgrade exists
 - 11: **end if**
-